

Shipping on the Air

Assignment #01 - Software Architecture and Platforms

A.Y. 2025/2026

Fabio Vincenzi

14 luglio 2026

Indice

1	Introduzione	2
2	Analisi	2
2.1	User Stories	2
2.2	Ubiquitous Language	3
2.3	Quality Attribute Scenarios	3
2.4	Acceptance Tests (BDD)	4
3	Design Strategico (DDD)	4
3.1	Subdomains	5
3.2	Bounded Contexts	5
3.2.1	God Classes evitare	5
3.3	Context Map	6
4	Design Tattico (DDD)	6
4.1	Building Blocks utilizzati	7
4.2	Marker Interfaces e Annotations	7
4.3	Domain Events	8
5	Architettura	8
5.1	Stile architetturale: Microservizi	8
5.2	Architettura interna: Clean/Hexagonal Architecture	9
5.2.1	La Dependency Rule	9
5.2.2	Fitness Functions (ArchUnit)	9
5.3	Comunicazione inter-servizio: Proxy Adapter	10
6	Implementazione	11
6.1	API REST	11
6.2	Esempio di flusso	11
7	Conclusioni	13

1 Introduzione

Il progetto *Shipping on the Air* realizza un sistema online per la consegna di pacchi tramite droni. L'utente può richiedere la spedizione di un pacco specificando luogo di ritiro, luogo di consegna, peso e tempistica (immediata o programmata), e monitorare in tempo reale lo stato della consegna.

Il processo di sviluppo segue i seguenti principi:

- **Domain-Driven Design** come approccio metodologico dall'analisi allo sviluppo;
- **Microservices** come stile architetturale;
- **Clean/Hexagonal Architecture** come struttura interna di ogni servizio.

2 Analisi

La Figura 1 mostra il diagramma dei casi d'uso del sistema.

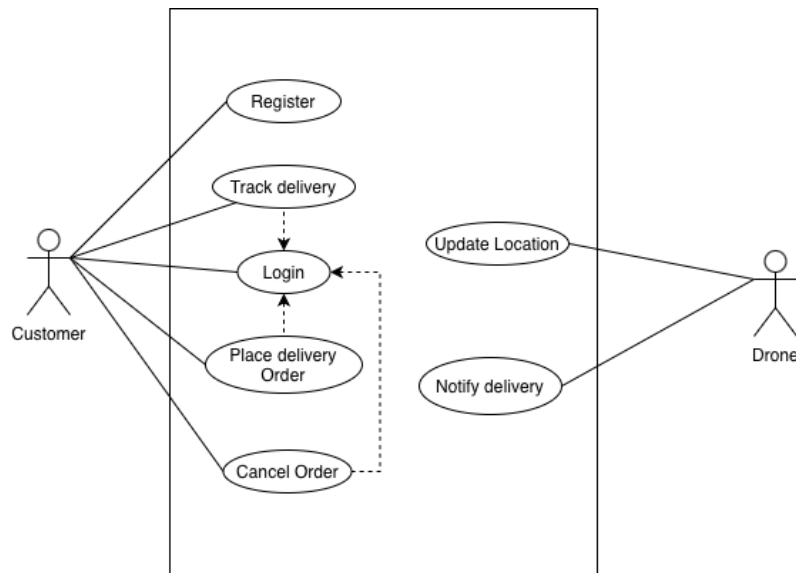


Figura 1: Diagramma dei casi d'uso

2.1 User Stories

Le user stories catturano i requisiti funzionali dal punto di vista degli attori del sistema.

ID	User Story
US-1	As a visitor, I want to register an account, so that I can use the delivery service.
US-2	As a customer, I want to log in, so that I can place and manage orders.
US-3	As a customer, I want to place a delivery order specifying pickup, delivery location and package weight, so that my package is delivered by drone.
US-4	As a customer, I want to choose immediate or scheduled delivery, so that it fits my needs.
US-5	As a customer, I want to see ETA before confirming, so that I can decide whether to proceed.
US-6	As a customer, I want to cancel a pending order, so that I can change my mind before dispatch.
US-7	As a customer, I want to track my delivery in real time, so that I know where my package is.
US-8	As a customer, I want to see the ETA, so that I know when to expect my package.
US-9	As a customer, I want to be notified when delivery is completed.
US-10	As the system, I want to automatically assign the nearest available drone to a confirmed order.
US-11	As the system, I want to receive real-time location updates from drones.
US-12	As the system, I want to release a drone after delivery completion.

Tabella 1: User Stories

2.2 Ubiquitous Language

L'*Ubiquitous Language* è il vocabolario condiviso tra esperti di dominio e sviluppatori, coerente all'interno di ogni Bounded Context.

Termine	Significato
Customer	Utente che richiede consegne tramite drone
Delivery Order	Richiesta di consegna: pacco, luoghi, tempistica, stato
Package	Oggetto da consegnare, caratterizzato da peso (kg)
Pickup Location	Luogo di ritiro (indirizzo + coordinate GPS)
Delivery Location	Luogo di consegna (indirizzo + coordinate GPS)
Drone	Veicolo aereo autonomo con capacità di carico e autonomia limitata
Delivery	Trasporto di un pacco dal pickup al delivery location tramite drone
Route	Percorso calcolato dal pickup al delivery location
ETA	Tempo stimato di arrivo
Tracking	Monitoraggio real-time di posizione drone e stato consegna

Tabella 2: Ubiquitous Language (estratto)

2.3 Quality Attribute Scenarios

I Quality Attribute Scenarios (QAS) descrivono i requisiti non-funzionali in modo misurabile, secondo il formato a 6 parti.

QAS	Attributo	Scenario
QAS-1	Performance	Il tracking restituisce la posizione GPS e ETA in meno di 1s, con aggiornamenti ogni 5s, sotto carico di 50 consegne attive.
QAS-2	Availability	Il servizio ordini accetta e conferma ordini con disponibilità del 99.9%.
QAS-3	Modifiability	L'integrazione di un nuovo modello di drone richiede modifiche in un solo servizio.
QAS-4	Scalability	Il raddoppio degli ordini giornalieri non causa degrado di performance grazie allo scaling orizzontale indipendente per servizio.

Tabella 3: Quality Attribute Scenarios

2.4 Acceptance Tests (BDD)

Gli acceptance test sono scritti in formato Gherkin e implementati con **Cucumber**. Ogni scenario è collegato a step definitions Java che testano direttamente il layer application, senza necessità di avviare i servizi HTTP.

```
Feature: Place delivery order

  Scenario: Successful immediate order
    Given I am a logged-in customer
    And at least one drone is available near the pickup area
    When I place a delivery order with:
      | pickup   | Via Roma 1, Cesena      |
      | delivery | Via Garibaldi 5, Forli  |
      | weight   | 2.5 kg                  |
      | when     | immediate                |
    Then a delivery order is created in CONFIRMED status
    And I receive a confirmation with estimated delivery time

  Scenario: Weight exceeds limit
    Given I am a logged-in customer
    When I place an order with a package weighing 50 kg
    Then the order is rejected
```

3 Design Strategico (DDD)

Il design strategico, secondo il Domain-Driven Design, identifica i subdomains nel problem space e definisce i Bounded Contexts nel solution space.

3.1 Subdomains

Subdomain	Tipo	Motivazione
Delivery Management	Core	Cuore del business: assegnazione droni, tracking, gestione consegne. Differenziatore competitivo.
Order Management	Supporting	Gestione ordini clienti. Necessario ma non differenziante.
Drone Management	Supporting	Gestione flotta droni, stato, posizione. Specifico del business ma non il differenziatore.
User Management	Generic	Registrazione e autenticazione. Soluzione standard.

Tabella 4: Subdomains identificati

3.2 Bounded Contexts

Ogni Bounded Context definisce i confini entro cui un modello e il suo Ubiquitous Language sono validi. Nel solution space, ogni BC corrisponde a un microservizio con proprio database e deploy indipendente.

Bounded Context	Servizio	Aggregate Root	Value Objects
Order Context	Order Service	Order	OrderId, Address, PackageInfo, OrderStatus
Delivery Context	Delivery Service	Delivery	DeliveryId, Route, DeliveryStatus
Drone Context	Drone Service	Drone	DroneId, Location, DroneStatus

Tabella 5: Bounded Contexts e relativi domain model

Il subdomain *User Management* (Generic) è gestito come modulo interno all'Order Service, non giustificando un servizio separato per il prototipo.

3.2.1 God Classes evitare

Lo stesso concetto ha rappresentazioni diverse nei diversi BC, evitando il problema delle God Classes:

- **Order Context:** conosce customer, package, prezzo, stato ordine, non sa nulla del drone o del percorso.
- **Delivery Context:** conosce pickup, delivery, drone assegnato (solo ID), route, ETA — non sa nulla del customer o del prezzo.
- **Drone Context:** conosce posizione, stato, capacità, non sa nulla di ordini o delivery.

3.3 Context Map

La Context Map (Figura 2) descrive le relazioni tra Bounded Contexts:

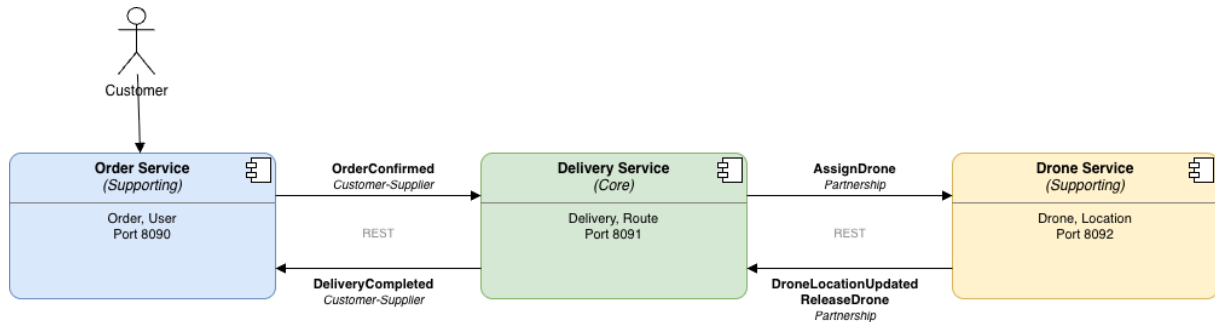


Figura 2: Context Map: relazioni tra Bounded Contexts

Relazione	Pattern	Comunicazione
Order → Delivery	Customer-Supplier	Order pubblica OrderConfirmed, Delivery reagisce
Delivery → Order	Customer-Supplier	Delivery pubblica DeliveryCompleted, Order reagisce
Delivery → Drone	Partnership	Delivery richiede assegnazione drone via REST
Drone → Delivery	Partnership	Drone pubblica DroneLocationUpdated

Tabella 6: Pattern di integrazione tra Bounded Contexts

In Assignment 1, tutte le interazioni sono implementate come chiamate REST sincrone tramite proxy adapter, senza event broker.

4 Design Tattico (DDD)

Il design tattico definisce i building blocks all'interno di ogni Bounded Context.

4.1 Building Blocks utilizzati

Building Block	Descrizione
Entity	Oggetto con identità persistente (es. <code>Order</code> , <code>User</code>)
Value Object	Oggetto senza identità, definito dai suoi attributi, immutabile (es. <code>Address</code> , <code>Location</code>)
Aggregate	Cluster di Entity e VO trattati come unità per data modification. Una transazione = un aggregate.
Aggregate Root	Unico punto di accesso all'aggregate dall'esterno (es. <code>Order</code> , <code>Delivery</code> , <code>Drone</code>)
Domain Event	Notifica immutabile di qualcosa di significativo accaduto nel dominio (es. <code>OrderConfirmed</code>)
Repository	Astrazione per accedere agli aggregate come collection in-memory

Tabella 7: Building Blocks DDD utilizzati nel progetto

4.2 Marker Interfaces e Annotations

I building blocks DDD e i concetti dell'architettura esagonale sono resi espliciti nel modulo `common/`.

I building blocks DDD sono **interfacce**, perché definiscono comportamento (es. `getId()`):

```
public interface Entity<T> { T getId(); }
public interface Aggregate<T> extends Entity<T> {}
public interface ValueObject {}
public interface DomainEvent {}
public interface Repository {}
```

I ruoli dell'architettura esagonale sono **annotazioni**, perché sono puri metadata senza comportamento:

```
public @interface InBoundPort {}
public @interface OutBoundPort {}
public @interface Adapter {}
```

Questa distinzione permette di verificare automaticamente con ArchUnit che le classi annotate `@InBoundPort` e `@OutBoundPort` risiedano nel layer application, e che le classi `@Adapter` risiedano nel layer infrastructure.

4.3 Domain Events

Bounded Context	Evento	Effetto
Order	OrderConfirmed	Delivery Service crea una nuova delivery
Order	OrderCancelled	Drone viene rilasciato
Delivery	DeliveryCompleted	Order passa a COMPLETED, drone rilasciato
Delivery	DeliveryFailed	Notifica di fallimento
Drone	DroneLocationUpdated	Delivery aggiorna posizione tracking
Drone	DroneStatusChanged	Aggiornamento stato flotta

Tabella 8: Domain Events e loro effetti cross-context

5 Architettura

5.1 Stile architetturale: Microservizi

Lo stile architetturale adottato è quello a **microservizi**, con domain partitioning guidato dal DDD. Ogni microservizio costituisce un *architecture quantum* indipendente: deploy indipendente, database proprio, alta coesione funzionale.

1. Identificazione delle system operations (dai requisiti funzionali)
2. Identificazione dei servizi (dai Bounded Contexts del DDD)
3. Definizione delle API di ogni servizio (commands, queries, events)

La Figura 3 mostra l'architettura complessiva del sistema.

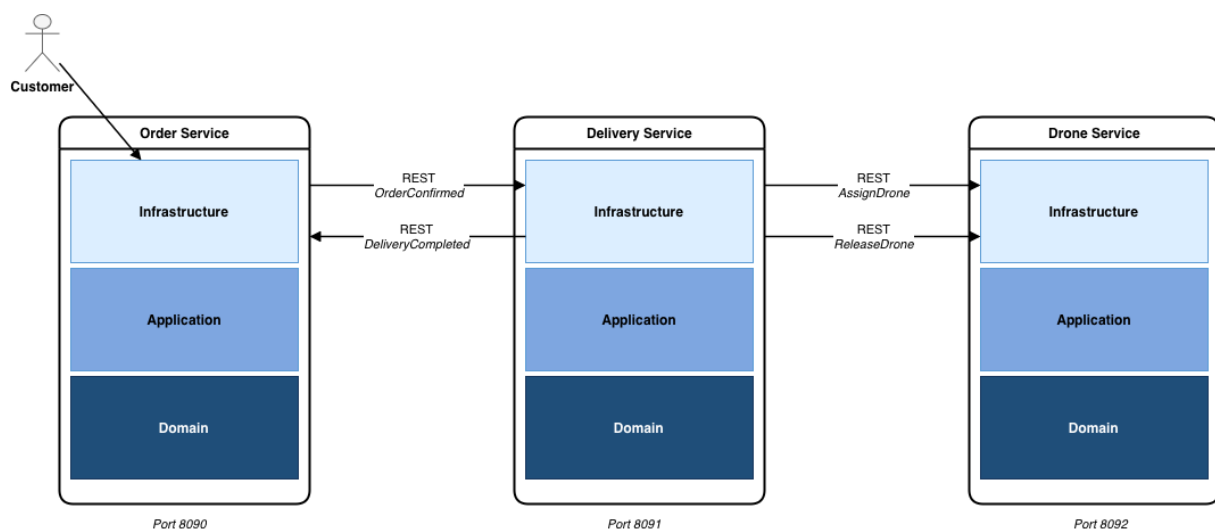


Figura 3: Architettura a microservizi con Clean Architecture interna e comunicazione REST

5.2 Architettura interna: Clean/Hexagonal Architecture

Ogni microservizio segue la **Clean Architecture**, con tre layer:

- **Domain** (cerchio interno) — Entities, Value Objects, Aggregates, Domain Events. Nessuna dipendenza verso l'esterno.
- **Application** (cerchio intermedio) — Use Cases (InBound Ports), interfacce Repository e Service Ports (OutBound Ports). Dipende solo dal domain.
- **Infrastructure** (cerchio esterno) — Controller REST (Adapter), implementazioni Repository in-memory (Adapter), Proxy Adapter verso altri servizi. Dipende da application e domain.

5.2.1 La Dependency Rule

Le dipendenze del codice sorgente puntano **solo verso l'interno**:

- domain non importa nulla da application o infrastructure
- application importa da domain ma non da infrastructure
- infrastructure importa sia da application che da domain

L'attraversamento dei confini avviene tramite **Dependency Inversion**: l'application layer definisce un'interfaccia (OutBound Port), l'infrastructure layer la implementa (Adapter).

La Figura 4 mostra il diagramma UML Component & Connector con la struttura interna di ogni servizio, evidenziando porte e adapter.

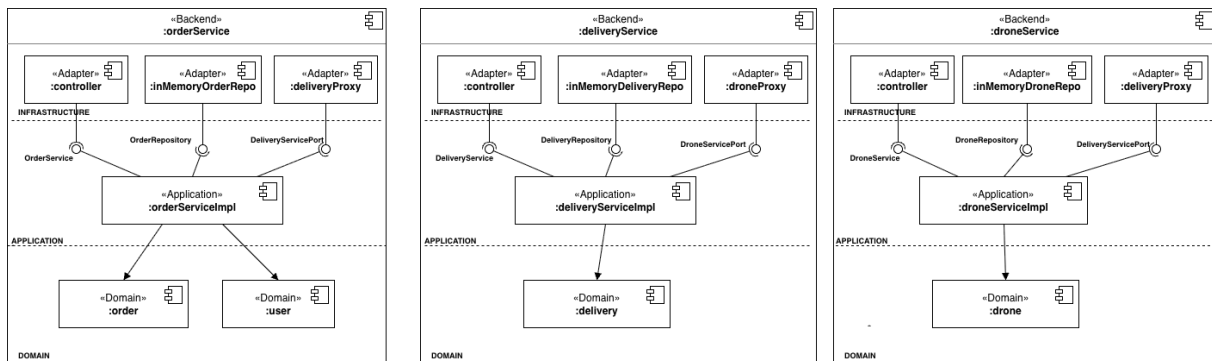


Figura 4: Diagramma C&C: struttura interna dei microservizi con porte e adapter

5.2.2 Fitness Functions (ArchUnit)

La Dependency Rule è verificata automaticamente tramite **ArchUnit** (fitness functions strutturali). Due test per ogni servizio:

Il test `cleanArchitecture()` verifica la layered architecture:

```
var layeredRule = layeredArchitecture()  
    .consideringAllDependencies()  
    .layer("Domain").definedBy("..domain..")  
    .layer("Application").definedBy("..application..")  
    .layer("Infrastructure").definedBy("..infrastructure..")  
    .whereLayer("Infrastructure").mayNotBeAccessedByAnyLayer()  
    .whereLayer("Application").mayOnlyBeAccessedByLayers("  
        Infrastructure")  
    .whereLayer("Domain").mayOnlyBeAccessedByLayers("Application"  
        , "Infrastructure");
```

Il test `hexagonalArchitecture()` verifica il posizionamento dei ruoli esagonali:

```
var portsRule = classes().that()  
    .areAnnotatedWith(InBoundPort.class).or()  
    .areAnnotatedWith(OutBoundPort.class)  
    .should().resideInAPackage("..application..")  
    .orShould().resideInAPackage("..domain..");  
  
var adaptersRule = classes().that()  
    .areAnnotatedWith(Adapter.class)  
    .should().resideInAPackage("..infrastructure..");
```

5.3 Comunicazione inter-servizio: Proxy Adapter

La comunicazione tra servizi segue il pattern **Proxy Adapter**:

1. L'application layer definisce un'interfaccia OutBound Port (es. `DroneServicePort`)
2. L'infrastructure layer implementa un adapter che effettua chiamate REST HTTP al servizio remoto (es. `DroneServiceProxy`)
3. Il servizio chiamante non conosce i dettagli di comunicazione, li decide l'adapter

Questo rispetta la Dependency Rule: il flusso di controllo va verso l'esterno (verso il servizio remoto), ma le dipendenze del codice puntano verso l'interno (l'adapter dipende dall'interfaccia definita nell'application layer).

6 Implementazione

6.1 API REST

Servizio	Metodo	Endpoint	Descrizione
Order	POST	/api/users/register	Registrazione utente
Order	POST	/api/users/login	Login utente
Order	POST	/api/orders	Creazione ordine
Order	POST	/api/orders/:id/confirm	Conferma ordine
Order	POST	/api/orders/:id/cancel	Cancellazione ordine
Order	GET	/api/orders/:id	Dettaglio ordine
Delivery	POST	/api/deliveries	Schedula delivery
Delivery	POST	/api/deliveries/:id/start	Avvia transito
Delivery	POST	/api/deliveries/:id/complete	Completa delivery
Delivery	GET	/api/deliveries/:id	Tracking delivery
Delivery	GET	/api/deliveries/order/:orderId	Cerca per ordine
Drone	POST	/api/drones	Registra drone
Drone	GET	/api/drones/available	Trova drone disponibile
Drone	POST	/api/drones/:id/location	Aggiorna posizione
Drone	POST	/api/drones/:id/release	Rilascia drone
Drone	GET	/api/drones/:id	Dettaglio drone

Tabella 9: API REST dei tre microservizi

Ogni servizio espone una specifica **OpenAPI 3.0** (/api/openapi.yaml) e un'interfaccia interattiva **Swagger UI** (/swagger-ui) che permette di esplorare e testare le API direttamente dal browser.

6.2 Esempio di flusso

La Figura 5 mostra il diagramma di sequenza del flusso principale.

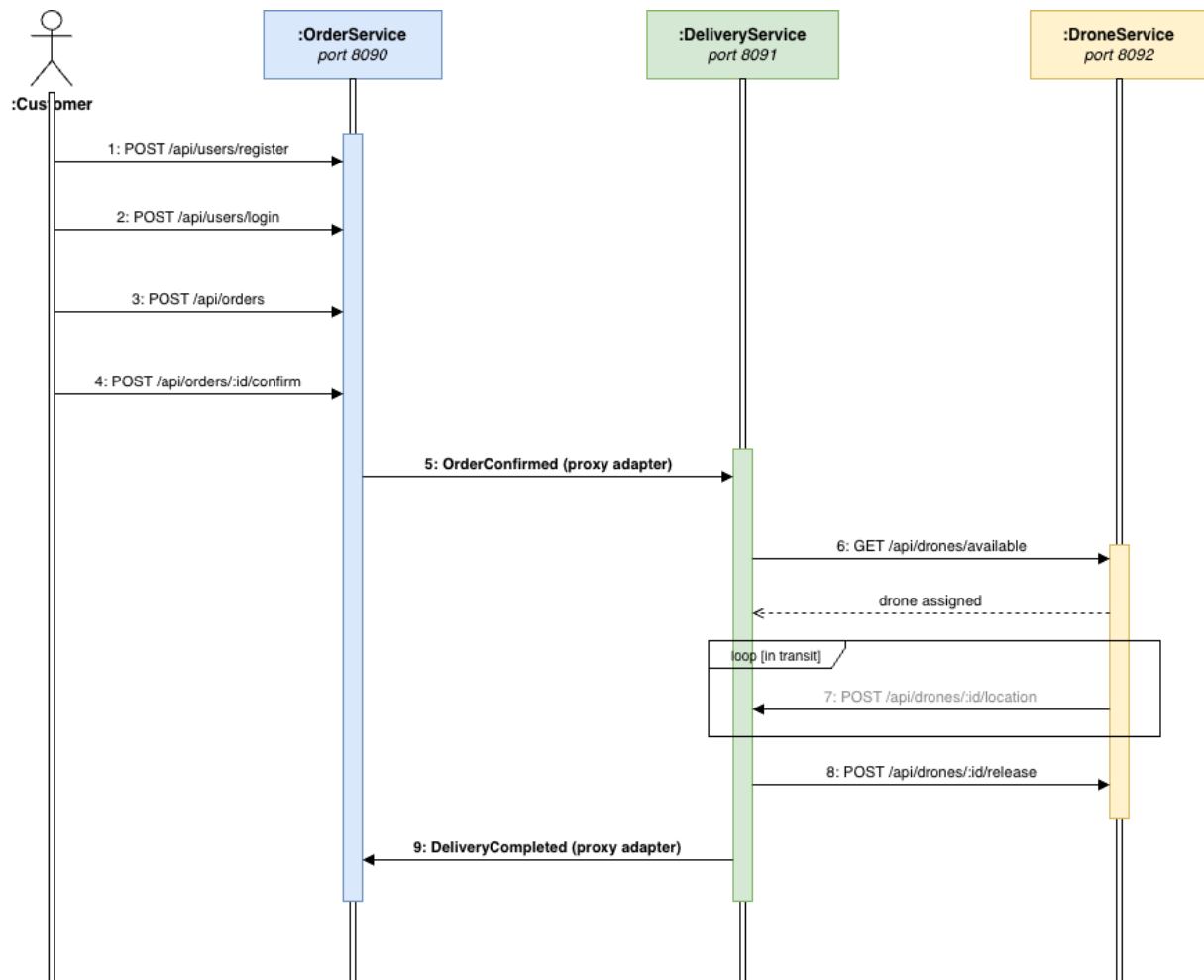


Figura 5: Diagramma di sequenza del flusso principale di consegna

Il flusso principale del sistema:

1. Il **Customer** registra un account e effettua il login (Order Service)
2. Il Customer crea un **Delivery Order** specificando pickup, delivery e peso (Order Service)
3. Il Customer **conferma** l'ordine (Order Service)
4. L'Order Service notifica il Delivery Service tramite **proxy adapter** (evento Order-Confirmed)
5. Il Delivery Service crea una **Delivery** e richiede un drone al Drone Service
6. Il Drone Service assegna il **drone più vicino** con capacità sufficiente
7. Il drone aggiorna la sua **posizione** durante il transito
8. Al raggiungimento della destinazione, la delivery viene **completata**
9. Il Delivery Service notifica l'Order Service (evento DeliveryCompleted) e rilascia il drone

7 Conclusioni

Il progetto dimostra l'applicazione integrata dei concetti studiati nel corso:

- Il **DDD strategico** ha guidato la decomposizione del dominio in subdomains e Bounded Contexts, determinando i confini dei microservizi.
- Il **DDD tattico** ha definito i building blocks interni a ogni servizio (Aggregates, Entities, Value Objects, Domain Events, Repositories).
- La **Clean/Hexagonal Architecture** ha strutturato ogni servizio in layer con la Dependency Rule, garantendo l'indipendenza del dominio dai dettagli tecnici.
- I **Proxy Adapters** hanno implementato la comunicazione inter-servizio rispettando il Dependency Inversion Principle.
- Le **fitness functions** (ArchUnit) verificano automaticamente il rispetto della Dependency Rule.

Ogni scelta architetturale è motivata dai Quality Attribute Scenarios: la decomposizione a microservizi favorisce *scalability* e *deployability*; la Clean Architecture favorisce *modifiability* e *testability*; il domain partitioning (vs. technical partitioning) allinea l'architettura al dominio di business.